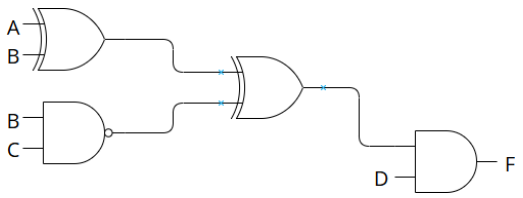
1. 아래의 조합 논리 회로 설명을 보고 문제를 해결하시오.

[회로 구성]



a. 회로의 최대 지연(Critical Path Delay)을 구하시오.

1. 모든 입력(A, B, C, D)에 대해 보수(Inverted)와 비보수 형태가 모두 추가 비용 없이 제공된다고 가정한다.
2. 오직 비보수 입력만 제공되며, 특정 경로를 위해 입력의 보수가 필요할 경우 인버터를 반드시 추가해야 한다고 가정한다. (이때 인버터가 추가된 경로의 지연을 포함하여 계산하시오.)

b. 입력 B의 신호 변화가 출력 F에 반영되기까지 거치는 모든 경로 중, 가장 긴 지연 시간을 구하시오.
(단, 모든 입력은 보수/비보수 모두 가능하다고 가정한다.)

2. n비트 이진수에서 1을 빼는 '감산기(Decrementer)' 회로를 설계하고자 한다. 이 회로는 n개의 동일한 모듈을 직렬로 연결하여 구현하며, 각 모듈은 해당 비트의 연산 결과와 상위 비트로 전달할 빌림수(Borrow)를 계산한다.

[설계 조건]

- i번째 모듈은 현재 비트 a_i와 하위 비트에서 올라온 빌림수 b_i를 입력으로 받습니다.
- 출력으로는 감산 결과 d_i와 다음 단계로 보낼 빌림수 b_{i+1} 을 내보냅니다.
- 최하위 비트($i = 0$)의 입력 빌림수 b_0은 1로 고정하여 전체 식 $A - 1$을 수행하게 합니다.
- 논리식: $d_i = a_i \oplus b_i$, $b_{i+1} = \bar{a}_i \cdot b_i$

a. 오직 NOR 게이트만을 사용하여 각 비트 모듈(Bit-slice)의 내부 회로를 설계하고 그리시오. (입력 a_i, b_i의 보수 신호는 제공되지 않는다고 가정하며, 필요시 NOR 게이트를 인버터로 활용하시오.)

b. n비트 전체 회로에서 최하위 빌림수 b_0이 입력되어 최상위 빌림수 b_n이 출력될 때까지의 총 전파 지연(Total Propagation Delay)을 n에 관한 식으로 나타내시오.

c. 연산 속도를 향상시키기 위해 '2비트 동시 처리 빌림수 예측(Lookahead Borrow)' 방식을 도입하고자 한다. b_{i+2} 를 a_{i+1}, a_i, b_i만을 이용하여 직접 계산하는 논리식을 유도하고, 이를 통해 n비트 연산 시 지연 시간이 어떻게 개선되는지 설명하시오.

3. Active-low 출력 디코더를 사용하는 다음 회로를 분석하고 입력 a, b, c에 따른 출력 F_1과 F_2에 대한 진리표를 그려라.

[회로 구성 상세]

- **디코더:** 3-to-8 라인 디코더이며, 모든 출력 라인은 **Active-low**(선택된 라인만 '0', 나머지는 '1')로 동작한다.
- **연결:**
  - 출력 **F_1**은 디코더의 출력 번호 **0, 2, 4, 6**이 **AND 게이트**의 입력으로 연결되어 있다.
  - 출력 **F_2**는 디코더의 출력 번호 **1, 3, 5, 7**이 **NAND 게이트**의 입력으로 연결되어 있다.
- **입력:** a가 MSB(가장 큰 비트), c가 LSB(가장 작은 비트)이다.

4. 4갈래 멀티플렉서(4-to-1 MUX)를 사용하여 다음 함수를 구현하라.

$$f(w, x, y, z) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$$

[설계 조건]

- 멀티플렉서의 선택 입력(S_1, S_0)으로 변수 **w와 x**를 사용하라. (w가 S_1, x가 S_0임)
- 데이터 입력 라인(I_0, I_1, I_2, I_3)에는 나머지 변수 y, z와 그 보수($\bar{y}, \bar{z}$), 또는 상수 0과 1 중 적절한 값을 연결하여 함수를 완성하고 회로도를 그려라.

문제 1. 논리회로 → 논리식

- 회로를 **왼쪽부터 게이트 순서대로** 식으로 변환
- AND → 곱, OR → 합, NOT → 바(bar) 처리

✅ **최종 논리식**

$$F = \left((A \cdot B) + (C \cdot D)\right)' \cdot D$$

(회로에 따라 정리하면 등가식 가능)

문제 2. Critical Path Delay

- **입력 → 출력까지 가능한 모든 경로**의 지연 합 계산
- **가장 큰 지연 경로**가 Critical Path

✅ **방법**

1. 각 경로의 게이트 지연을 모두 더함
2. 최댓값 선택

✅ **답**

- **Critical Path**: 가장 많은 게이트를 거치는 경로
- **총 지연 시간**: (주어진 게이트 지연값 합)

문제 3. Decrementer (Borrow 포함)

- Decrementer = 입력에서 **1을 빼는 회로**
- LSB부터 연산, Borrow 전파 고려

✅ **출력 비트**

$$S_i = A_i \oplus B_i$$

✅ **Borrow 발생 조건**

$$B_{i+1} = A_i' \cdot B_i$$

→ 이전 비트가 0이고 Borrow가 들어오면 다음 Borrow 발생

문제 4. Active-low + 4:1 MUX

- 주어진 함수:

$$f(w, x, y, z) = \Sigma m(0, 2, 5, 7, 8, 10, 13, 15)$$

✅ **풀이 절차**

1. 선택선 = y, z
2. 각 yz 조합마다 f 값 정리
3. Data 입력 $I_0 \sim I_3$ 결정
4. Active-low → 입력/출력 반전 고려

✅ **결과**

- 4:1 MUX 하나로 구현 가능
- Data 입력은 w, x 또는 상수(0/1) 조합으로 구성

Gemini를 이용해 문제를 생성하고 M365 copilot으로 풀이함